

Counting jobs, not shots: batched pattern search for the quantum approximate optimization algorithm on cloud-accessed hardware

Muhammad Faryad^{1,*}

¹*Department of Physics, Lahore University of
Management Sciences (LUMS), Lahore 54792, Pakistan*

(Dated: August 21, 2026)

Abstract

The cost of a variational quantum algorithm is almost always reported in circuit evaluations or in shots. On a cloud-accessed processor neither is the quantity a user actually spends: the unit that is charged and the unit that is queued is the *submitted job*. We measure that unit on IBM’s `ibm_fez`. A job carries a fixed overhead independent of its contents — of order three seconds of metered processing time by IBM’s own documented estimator, and, in free-tier job mode, a measured round trip that never once fell below 10.7s across the 193 jobs of this study, with a median of 19.2s and a tail reaching 1454s. Under this cost model an optimizer should be judged by what it learns per job, and a serial optimizer that spends one job per function evaluation pays that overhead $4p + 2$ times over for what a batched iteration buys once. We therefore run a *batched pattern search*, whose entire iteration is known before any function value returns and which therefore probes $4p + 2$ points of the variational landscape in a single job, against COBYLA under an identical budget of jobs *and* shots. On four nested cardinality-constrained portfolio instances of 6, 8, 10 and 12 assets, solved on `ibm_fez` and verified against exhaustive enumeration, the batched search reaches after *one* job a parameter quality that the serial optimizer needs three to twelve jobs to match, an ordering that also survives four independent initializations at $n = 8$. The device ranks feasible portfolios in near-agreement with the exact ordering (Spearman ρ_s from -0.97 to -0.45) and returns the true optimum as its most probable feasible outcome at $n = 6, 8$ and 10, and as its second most probable of 924 candidates at $n = 12$, where shot noise on ~ 30 counts no longer resolves the leading few; a $\gamma = 0$ control of identical depth shows $\rho_s \approx 0$, confirming that the ordering is not a device artefact. A single two-parameter law, $\eta = 1 - \exp[-(0.191 + 1.350 \times 10^{-3} N_{2q})]$, reproduces the effective depolarizing strength at all four sizes, and a 25-job depth scan at classically optimal parameters locates the device’s optimal circuit depth at $p = 2-3$, beyond which added layers cost more fidelity than they buy. The argument is not specific to QAOA: any variational algorithm executed through a queue faces the same asymmetry between what a job costs and what a job can carry.

* muhammad.faryad@lums.edu.pk

I. INTRODUCTION

A variational quantum algorithm is a classical optimizer wrapped around a quantum subroutine, and the conventional way to report its cost is to count the calls it makes to that subroutine — function evaluations, circuits, or shots. That accounting is natural if the quantum computer sits on the desk. Almost none of them do. The overwhelming majority of variational experiments are executed on processors reached through a cloud queue, and there the resource a user spends is neither a shot nor an evaluation but a *job*: a bundle of circuits submitted as one unit, charged as one unit, and queued as one unit.

The distinction is not bookkeeping. A job has a fixed cost that its contents do not change. We measure this cost directly in Sec. IV: on IBM’s 156-qubit Heron r2 processor `ibm_fez`, the metered quantum-processing time of a job is

$$T_{\text{QPU}} = 3.00 \text{ s} + \tau(D_{2q}) \times N_{\text{shots}}, \quad (1)$$

with τ of order 0.4 ms. A job carrying a single circuit at 5000 shots therefore spends more than half its metered budget on the fixed term. Wall-clock time is more lopsided still, and unlike the metered figure it is something we measured directly: over the 193 jobs reported here the median round trip was 19.2 s, the fastest ever observed was 10.7 s, and the tail reached 1454 s. Queueing and turnaround on shared quantum hardware have been characterized before [1–3], and the implications for stochastic optimizers have been analyzed theoretically [4]; what has not been common practice is to design and benchmark the optimizer with the job as the explicit unit of account.

Doing so changes which optimizer one should reach for. A job may carry many circuits, and in Qiskit’s primitive interface a single *primitive unified bloc* (PUB) may carry an arbitrary number of parameter vectors for one parameterized circuit. An optimizer can use that capacity only to the extent that it can name points it wants evaluated *before* it sees any of the values, and methods differ enormously in how many points that is. At one extreme, COBYLA [5], Nelder–Mead [6] and Powell [7] are sequential by construction once past their initial simplex or direction set: each next point depends on the value just returned, so each evaluation is its own job. In the middle, SPSA [8] names *two* points per iteration — the $\pm c_k \Delta_k$ perturbations are both known before either value returns — and a parameter-shift gradient [9, 10] names all of its evaluations at once; both are batchable, and we say so because the point is sometimes overlooked. At the far end sits pattern search [11–13], whose

entire iteration — the incumbent, a fixed stencil of trial points around it, and, in the variant used here, one momentum probe — is determined before any value returns. For a depth- p QAOA ansatz with $2p$ parameters that is $4p + 2$ landscape points in a single job. The claim of this paper is not that batching is exotic; it is that points-per-round-trip is the axis along which optimizers should be compared on queued hardware, and that it is rarely the axis reported.

What this paper contributes.

1. **An explicit cost model for a hardware job** (Sec. IV), instantiated from IBM’s documented estimator for `ibm_fez` and used to budget every run in advance, together with the measured wall-clock cost of all 193 jobs. Both expose a fixed per-job overhead that the usual shot-based accounting hides, and we are explicit about which of the two is measured: the wall clock is, the metered time is not.
2. **A job-budgeted benchmarking protocol** (Sec. VI) in which the batched and the serial optimizer are given an identical number of jobs *and* an identical number of shots, so that the only difference between the two arms is whether a job’s shot budget is spent on one point of the landscape or spread across $4p + 2$ of them. The intent is to remove shot count as an explanation, so that the comparison isolates allocation; Sec. VI discusses how much the serial arm can actually do with its $(4p + 2)$ -fold surplus, which on our own tuning data is little.
3. **A hardware demonstration at four sizes** (Sec. VII) on cardinality-constrained portfolio selection with $n = 6, 8, 10, 12$ assets, each small enough that the exact optimum and the exact ranking of every feasible portfolio are obtained by enumeration, so no claim rests on a heuristic reference.
4. **Controls that are usually missing**: a $\gamma = 0$ job of identical gate count and depth whose ideal output is exactly $|+\rangle^{\otimes n}$, which measures the noise floor; and a repeat of the first optimization job at the end of the run, which measures device drift across the comparison.
5. **A device characterization by-product**: the effective depolarizing strength inferred from the measured distributions follows a single two-parameter law in the two-qubit gate count across all four instances (Sec. VIID).

6. **A measured depth optimum** (Sec. VII E). Because the noiseless landscape is exactly optimizable at these sizes, the depth–noise trade-off can be measured with the optimizer removed: one job per (n, p) at classically optimal parameters. The device optimum is $p = 2\text{--}3$, and reoptimizing against the noise model at fixed depth recovers none of the loss beyond it.
7. **A replicate study** (Sec. VII F) repeating the 8-asset comparison from four initializations, which puts a spread on the headline convergence claim.

Why this matters beyond QAOA. Nothing in the argument uses a property of QAOA. It uses two facts: that a job costs something before it carries anything, and that some optimizers can name a batch of query points in advance while others cannot. Both hold for every variational algorithm executed through a queue. Variational eigensolvers sweep parameters the same way; quantum machine learning trains parameterized circuits the same way; quantum kernel methods evaluate a Gram matrix whose entries are mutually independent and therefore perfectly batchable; hardware characterization, error-mitigation calibration and pulse tuning are parameter sweeps by construction. The same asymmetry appears on trapped-ion and neutral-atom platforms reached through the same kind of cloud interface. In every case the design question posed here is the same one: given that you will pay a fixed price J times, what is the most informative set of M points to put inside each of those J payments? Pattern search is one answer, and a deliberately simple one; model-based trust-region methods [14], multistart schemes [15] and parameter-transfer strategies [16–18] are others, and all of them become more attractive, not less, when the job rather than the evaluation is the thing being counted.

The remainder of the paper is organized as a route that can be followed end to end. Section II states the optimization problem and its Ising form; Sec. III constructs the circuit; Sec. IV measures what a job costs; Sec. V states the optimizer; Sec. VI fixes the protocol and its controls; Sec. VII reports the hardware results; Sec. VIII discusses limitations and consequences. Appendix A records how every hyperparameter was chosen on a noise model *before* any quantum processing time was spent, and Appendix B describes the released artifact.

II. THE PROBLEM AND ITS ISING FORM

We solve the discrete, cardinality-constrained form of Markowitz mean–variance portfolio selection [19], a standard testbed for quantum optimization [20–23]. Let $x_i \in \{0, 1\}$ indicate that asset i is held, $\mathbf{r}$ be the vector of expected returns and Σ the covariance matrix. With a hard budget of exactly B assets, the cost function is

$$C(\mathbf{x}) = \lambda \mathbf{x}^\top \Sigma \mathbf{x} - \mathbf{r}^\top \mathbf{x} + \mu \left(\sum_i x_i - B \right)^2, \quad (2)$$

the last term being the usual quadratic penalty that converts the constraint into a QUBO [24, 25]. Two coefficients in Eq. (2) deserve more care than they usually receive, because both can silently trivialize the problem.

The risk aversion λ . If λ is small the quadratic term does nothing and the “optimization” is a sort by expected return, which QAOA would then be credited with solving. We remove the choice by fixing λ so that, across the feasible set, the range of the risk term equals the range of the return term,

$$\lambda = \frac{\max_{\text{feas}}(\mathbf{r}^\top \mathbf{x}) - \min_{\text{feas}}(\mathbf{r}^\top \mathbf{x})}{\max_{\text{feas}}(\mathbf{x}^\top \Sigma \mathbf{x}) - \min_{\text{feas}}(\mathbf{x}^\top \Sigma \mathbf{x})}. \quad (3)$$

Every instance below therefore has a risk-to-return spread ratio of exactly 1.000, and the optimum is genuinely a trade-off rather than a ranking.

The penalty μ . Too small and the global minimum of Eq. (2) is infeasible; too large and μ dominates every Hamiltonian coefficient, the ZZ couplings become nearly uniform, and the ansatz spends its layers enforcing the Hamming weight instead of discriminating among feasible portfolios. We scanned μ against the exactly optimized ideal $p = 2$ QAOA (Appendix A) and took the maximum of that scan, which lies at $\mu \approx 0.05 \times$ the spread of the penalty-free objective over the feasible set. The resulting values are comfortable: at $n = 6$ and 8 the penalty-free objective already has a feasible minimum, so any $\mu > 0$ suffices, and at $n = 10$ and 12 the smallest penalty that makes the ground state feasible is 5.4 and 3.6, against the chosen 12 and 13.

Under $x_i = (1 - z_i)/2$, Eq. (2) becomes an Ising Hamiltonian

$$\tilde{H} = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j, \quad C = s \tilde{H} + c, \quad (4)$$

where the identity term has been removed and the coefficients rescaled so that $\max(|h|, |J|) = 1$. The rescaling is not cosmetic: the pattern-search step size and the shot-noise tolerance of

TABLE I. The four nested instances. $r(\text{uniform})$ is the expected approximation ratio of the uniform distribution over the feasible set, the baseline any method must beat. All quantities are exact, from enumeration of all 2^n bitstrings.

n	B	λ	μ	$\binom{n}{B}$	$C_{\min}$	$C_{\max}^{\text{feas}}$	$r(\text{uniform})$
6	3	1.771	8	20	-136.8	21.1	0.451
8	4	1.653	9	70	-161.1	23.3	0.456
10	5	1.553	12	252	-219.4	23.7	0.490
12	6	1.360	13	924	-243.3	26.0	0.505

Sec. V are both expressed in these units, and without it they would have to be retuned for every instance. In these units the uniform superposition has $\langle \tilde{H} \rangle = 0$ exactly, so a run that ends at a positive value has done worse than doing nothing. Every notebook asserts that the diagonal of Eq. (4) reproduces Eq. (2) on all 2^n bitstrings before any circuit is built.

The four instances are *nested*: instance n consists of the first n assets of a fixed twelve-asset table of large-capitalization US equities, with $B = n/2$. Any change in the results across n is therefore a property of the problem growing, not of four unrelated problems. Their parameters and exact solutions are given in Table I.

We report three quantities throughout. The *feasible probability mass* is the fraction of shots with Hamming weight B ; post-selecting on it is free error mitigation, since every other bitstring is provably an error. The *expected approximation ratio conditioned on feasibility* is

$$r = \frac{C_{\max}^{\text{feas}} - \langle C \rangle_{\text{feas}}}{C_{\max}^{\text{feas}} - C_{\min}}, \quad (5)$$

an average over the whole post-selected distribution and therefore, unlike a best-of- N figure, immune to inflation by taking more shots. The third is $p(\text{optimum} \mid \text{feasible})$, reported as a multiple of the uniform value $1/\binom{n}{B}$.

The optimizer, however, does not minimize Eq. (5); it minimizes the conditional value at risk [26], CVaR_α , the mean energy of the lowest α fraction of the sampled probability mass, at $\alpha = 0.5$. On a noisy device the mean energy is dominated by the depolarized background while the tail still carries the signal; $\alpha = 0.5$ was selected in simulation (Appendix A) as the best compromise between that effect and the shot noise of a short tail. Related tail-based and filtering objectives are discussed in Ref. [27].

III. CIRCUIT SYNTHESIS ON A CHAIN

The cost Hamiltonian of Eq. (4) is a *complete* graph: every pair of assets is coupled. The processor is heavy-hex. Handing the ansatz to a preset transpiler therefore produces a routing solution whose depth is set by a SWAP heuristic and changes with the random seed — an unattractive property when the circuit is the independent variable of an experiment.

We instead place the n qubits on a single physical chain and realize all $\binom{n}{2}$ couplings with an odd-even transposition network [28–30]: at each of n steps, disjoint neighbouring pairs interact and then swap, so that after n steps every pair has met exactly once. Writing $\text{RZZ}(\theta) \cdot \text{SWAP}$ by hand as

$$\text{cx}(a, b) \text{ rz}(\theta, b) \text{ cx}(b, a) \text{ cx}(a, b), \quad (6)$$

which is exact up to a global phase, costs three entangling gates per pair. The resulting gate count is deterministic,

$$N_{2q} = 3p \binom{n}{2}, \quad D_{2q} = 3pn, \quad (7)$$

independent of transpiler seed, and the final measurement permutation is tracked so that classical bit i always carries logical qubit i .

Emitting Eq. (6) explicitly rather than leaving it to the compiler matters: Qiskit’s two-qubit block consolidation cannot fuse gates carrying unbound `Parameters`, so a transpiled $\text{RZZ} \cdot \text{SWAP}$ costs five entangling gates instead of three. Table II and Fig. 1 compare the hand-built network against the preset pass manager at optimization level 3: the network uses 1.28–1.37 times fewer two-qubit gates at 2.4–2.8 times lower two-qubit depth. The chain itself is chosen at run time by a beam search over simple paths of the coupling map that minimizes $\sum -\log(1 - \varepsilon)$ over the CZ and readout errors of the path, so that the circuit sits on the best line the device has on the day, and the notebook asserts that the pass manager did not move it. Each circuit is verified against the exact QAOA state vector before submission.

IV. WHAT A JOB COSTS

IBM defines the metered usage of a job as “the amount of time a QPU is locked to execute workloads” [31]. The documentation gives an estimation formula of the form (*per-sub-job*

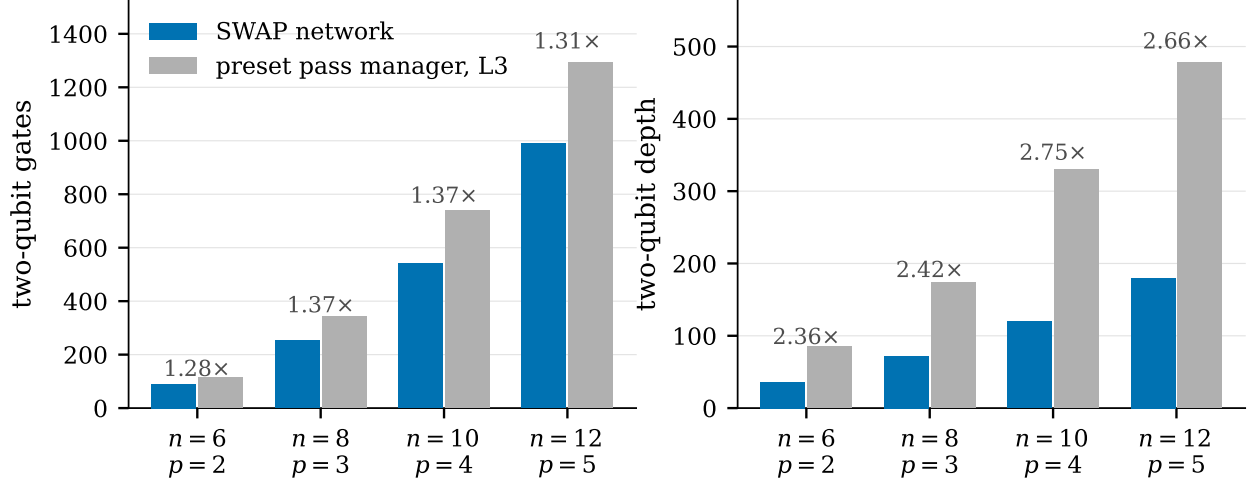


FIG. 1. Two-qubit gate count and two-qubit depth of the swap-network synthesis against Qiskit’s preset pass manager at optimization level 3, for the four circuits actually submitted. Labels give the ratio.

TABLE II. Circuit resources on `ibm_fez`. The swap-network columns follow Eq. (7) exactly; the preset columns depend on the transpiler seed.

n	p	swap network		preset, level 3		ratio	
		N_{2q}	D_{2q}	N_{2q}	D_{2q}	gates	depth
6	2	90	36	115	85	1.28	2.36
8	3	252	72	344	174	1.37	2.42
10	4	540	120	738	330	1.37	2.75
12	5	990	180	1293	478	1.31	2.66

overhead) + (*rep_delay* + *circuit length*) × (*number of executions*), with an overhead of roughly two seconds. Every job of this study submitted a single PUB in job mode — no `Session`, no `Batch` — which is the access mode available on the free tier [32].

Instantiating that formula for `ibm_fez`, with its 250 μ s default repetition delay and the gate durations of its calibration data, and rounding the per-sub-job overhead up to three seconds for budgeting, gives

$$T_{\text{QPU}} \simeq 3.00 \text{ s} + (352.6 + 0.267 D_{2q}) \mu\text{s} \times N_{\text{shots}}, \quad (8)$$

with D_{2q} the two-qubit depth. The per-shot coefficient decomposes into a depth-independent

352.6 μs — the repetition delay plus reset, readout and payload overhead — and a 0.267 μs penalty per unit of two-qubit depth; for the circuits run here it ranges from 362 to 401 μs . Equation (8) is what the ledger of Appendix B used to budget every run in advance, and it is why each notebook’s cost was known to the second before it was launched.

A caveat that must be stated plainly. Every job of this study also attempted to record the runtime’s own reported usage, and that call failed silently on all 193 jobs: the key path used to read `job.metrics()` does not exist in the client version we ran, and the surrounding `try/except` fell back to the predicted value. We discovered this only when auditing the released data, in which the recorded and predicted usage are consequently bit-identical. **Every figure for metered QPU time in this paper is therefore Eq. (8) evaluated on the job’s contents, not a measurement of the device.** It is a faithful instantiation of IBM’s documented estimator and we expect it to be close, but we cannot claim it as data, and no conclusion here should be read as resting on it. The failure is the fourth entry in the list of silent errors in Appendix B, and it is recorded there in the same spirit as the other three.

What *was* measured, on every job, is wall-clock time from submission to result. That is the quantity the argument actually needs, and it is the harsher of the two. Across the 193 jobs of this study the median round trip was 19.2s, the fastest ever observed was 10.7s, and the distribution has a heavy tail reaching 1454s (Fig. 2). Even the smallest jobs submitted — 5 112 shots, at the shallowest circuit — took a median of 14.4s to come back. There is, in other words, a floor of order ten seconds on a round trip that no reduction in a job’s contents removes, and the aggregate over the study is 143 minutes of wall clock for 34 minutes of (modelled) quantum processing.

The consequence for a variational loop is the same under either accounting. An optimizer that submits one job per function evaluation pays the per-job overhead — of order three seconds metered, of order ten to twenty seconds in wall clock — once per evaluation, before a single shot is taken. A batched iteration that puts M parameter vectors into one PUB pays it once for all M . At the settings used below M ranges from 10 to 22, so the serial arm spends between 10 and 22 times as much per-job overhead for the same number of landscape points examined. Table III gives the full ledger.

Two scope limits belong here rather than in the discussion. First, wall-clock time is not a property of the device or of the algorithm: it depends on other users’ load and on fair-

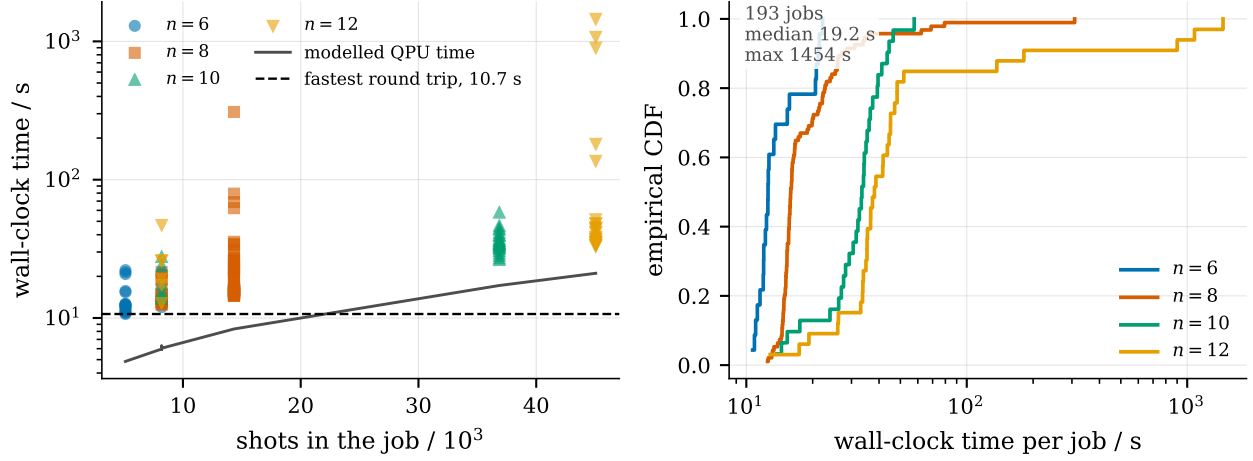


FIG. 2. The cost of a job on `ibm_fez`, from the 193 jobs of this study. **Left:** measured wall-clock time from submission to result against the number of shots the job carried. The dashed line is the fastest round trip observed, 10.7 s; no job returned faster, whatever it contained. **Right:** the empirical distribution of the same times, by instance size. Both panels are measured; the modelled metered time of Eq. (8) is shown for scale only.

share priority, and our own per-run ratios vary by a factor of four. It should be read as a characterisation of the access mode, not as a constant. Second, and more important, that access mode is job mode on the free tier. IBM’s **Batch** and **Session** modes exist precisely to amortise queueing across a sequence of jobs, and a user with access to them would see a much smaller wall-clock penalty — though the per-job metered overhead of Eq. (8) is charged in all three modes, and in **Session** mode the meter runs on elapsed time whether or not a circuit is executing [31]. The argument of this paper is therefore strongest for the open-plan user and weakest for the reserved-capacity one; it does not vanish for either.

V. BATCHED PATTERN SEARCH

Pattern search, also called compass or direct search, replaces derivatives with a fixed stencil of trial points around an incumbent, accepting a move only when a trial point improves on the incumbent and contracting the stencil when none does [11, 12]. Its relevant property here is structural rather than statistical: the entire iteration is determined by the incumbent and the current radius, so every point can be named before any value comes back.

Algorithm 1 states the variant used. For $2p$ parameters an iteration consists of the

TABLE III. Resource ledger for the four main runs. “QPU” is Eq. (8) evaluated on each job’s contents — modelled, not measured, for the reason given in the text — while “wall” is the measured sum of submission-to-result times. The two follow-up studies added 91 further jobs, giving 193 jobs for the study as a whole. Every job identifier is in the released data.

n	p	shots/cand.	shots/job	jobs	total shots	QPU / s	wall / s	ratio	median wall / s
6	2	512	5,120	20	111,600	100	292	2.9	13
8	3	1024	14,336	24	325,612	193	942	4.9	23
10	4	2048	36,864	28	946,160	448	968	2.2	34
12	5	2048	45,056	30	1,241,066	587	4688	8.0	38
total				102	2,624,438	1329	6890	5.2	

incumbent (row 0), the $2 \times 2p$ axial probes at radius ρ , and, when the previous iteration moved, one momentum probe continuing along the accepted step: at most $4p + 2$ points, all submitted as the rows of a single PUB in a single job.

Two details are not decoration. The acceptance test requires an improvement larger than ϵ , half the standard error of the measured energy of the incumbent row. Without it, at the shot counts used here, the incumbent moves on nearly every iteration and the search chases shot noise rather than the landscape; the same measurement that supplies the incumbent value supplies σ , so the tolerance costs nothing. The momentum probe exploits the fact that the batch has a spare row whenever the previous iteration moved, and extrapolates along the accepted direction; it is the cheapest available approximation to a line search.

Because the number of candidates and the shots per job are both fixed in advance, the entire quantum cost of a run of J iterations is known before it starts, to the millisecond, from Eq. (8). That is a practical property no serial optimizer has: COBYLA’s evaluation count is data-dependent and must be truncated at an arbitrary point.

The initial point is the linear ramp $\beta_k = \Delta t (1 - (k - \frac{1}{2})/p)$, $\gamma_k = -\Delta t k/p$ with $\Delta t = 0.5$, an adiabatically motivated schedule of the kind introduced in Ref. [33] and shown to avoid the local minima that plague random initialization [34]; warm-starting from a classical relaxation is a further option we did not use [35]. The sign of γ follows the convention of Eq. (4).

Algorithm 1. Batched pattern search. One iteration = one job.

Require: incumbent $\mathbf{x}_0$, radius ρ , shot budget S_{job} per job

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0, \quad \mathbf{x}_{\text{prev}} \leftarrow \mathbf{x}_0$ 
2: for  $j = 1 \dots J$  do  $\triangleright J = \text{the job budget}$ 
3:    $\mathcal{C} \leftarrow \{\mathbf{x}\} \cup \{\mathbf{x} \pm \rho \mathbf{e}_i\}_{i=1}^{2p}$ 
4:   if  $\mathbf{x} \neq \mathbf{x}_{\text{prev}}$  then
5:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{2\mathbf{x} - \mathbf{x}_{\text{prev}}\}$   $\triangleright \text{momentum probe}$ 
6:   end if
7:   submit one job: one PUB,  $|\mathcal{C}|$  parameter rows,  $S_{\text{job}}/|\mathcal{C}|$  shots each
8:    $v_c \leftarrow \text{CVaR}_\alpha$  of the distribution measured for  $c \in \mathcal{C}$ 
9:    $\sigma \leftarrow$  standard deviation of  $\tilde{H}$  under row 0
10:   $\epsilon \leftarrow \frac{1}{2} \sigma / \sqrt{S_{\text{job}}/|\mathcal{C}|}$   $\triangleright \text{shot-noise tolerance}$ 
11:   $c^* \leftarrow \arg \min_c v_c$ 
12:  if  $c^* \neq \mathbf{x}$  and  $v_{c^*} < v_{\mathbf{x}} - \epsilon$  then
13:     $\mathbf{x}_{\text{prev}} \leftarrow \mathbf{x}, \quad \mathbf{x} \leftarrow c^*, \quad \rho \leftarrow \min(1.3\rho, \pi/2)$ 
14:  else
15:     $\rho \leftarrow \max(0.6\rho, 0.02)$ 
16:  end if
17: end for

```

VI. PROTOCOL AND CONTROLS

Each instance is run as a self-contained experiment with two arms under an identical budget.

Matched budget. Both arms receive the same number of jobs J and the same shots per job S_{job} . The batched arm spends a job on $M = 4p + 2$ parameter vectors at S_{job}/M shots each; the serial arm spends a job on one parameter vector at the whole S_{job} . The two arms therefore consume identical quantum resources by both metrics, and the difference under test is the allocation.

We should be careful about how much this constraint costs the batched arm. Nominally it hands the serial arm a M -fold advantage in shot noise per evaluation. In practice the pre-registered tuning study of Appendix A found final quality flat in shots per candidate

between 512 and 2048, so a serial evaluation at 14 336 or 45 056 shots is far into the regime where extra shots buy nothing, and the surplus cannot be spent anywhere else. The honest reading is therefore that the shot-matching constraint is close to a nullity for the serial arm and the comparison reduces to M points per job against one. We keep it because it forecloses the objection, not because it is a meaningful handicap.

One asymmetry runs the other way and should also be named. The batched arm’s M rows execute inside a single job under one calibration snapshot, so comparisons among them are drift-free; COBYLA’s successive evaluations sit in different jobs, minutes apart, and carry inter-job drift on top of shot noise. That is a real advantage of batching, and it is part of what the method buys, but it means “the only difference is the allocation” would be too strong a statement.

The serial optimizer. COBYLA [5] with ρ_{beg} equal to the pattern-search radius. It was chosen because it was the strongest of COBYLA, SPSA, Nelder–Mead and Powell in the pre-registration study of Appendix A at every size, over the range of job budgets used here; the control should be the best available serial method, not a straw man. (SPSA is stronger at a budget of two jobs, and Powell overtakes beyond about forty jobs; neither is the regime studied.) This is consistent with published comparisons of optimizers under noise [36, 37].

Error suppression. Dynamical decoupling [38, 39] on idle qubits and Pauli twirling of gates and measurements [40] were enabled for every job; neither consumes extra shots. No zero-noise extrapolation or probabilistic error cancellation [41] was used, so the measured distributions are raw device output apart from these two settings; the only mitigation applied anywhere in this paper is post-selection on Hamming weight, which is free.

The $\gamma = 0$ control. After the optimization arms, one job is submitted at the batched arm’s best β with γ set to zero. The circuit has *identical* gate count and depth, but its ideal output is exactly $|+\rangle^{\otimes n}$. Any structure that survives in it is the device’s noise floor and not something the algorithm found. This is the reference against which every claim below should be read.

The drift control. The two arms necessarily run one after the other, so a device that drifts over the half hour would flatter whichever arm ran in better conditions. The last job of every run repeats the *first* batched iteration verbatim. The mean shift in CVaR across the run was -0.034, +0.043, -0.050 and +0.028 for $n = 6, 8, 10, 12$ — of the order of the shot noise, and of both signs, so no drift correction is applied.

Two views of quality. Because $n \leq 12$, the noiseless output distribution of any parameter vector is computable exactly and cheaply. We therefore report, alongside what the device measured, an oracle curve: the exact noiseless approximation ratio, evaluated classically, of the best point each arm holds after each job. For the batched arm “holds” means the *incumbent* — the point that has survived the acceptance test of Algorithm 1 — and the curve is the running maximum of that incumbent’s noiseless ratio over jobs. For the serial arm it is the running maximum over the points COBYLA has evaluated, which is what COBYLA returns. Two things about this metric must be stated. It is deliberately generous to the serial arm, since the batched arm is credited only with points its acceptance rule kept, while COBYLA is credited with the best of everything it touched. And it is an oracle: neither optimizer can compute it, and neither could at a size worth solving. It is a diagnostic of the search, not a quantity either method returns, and it is why the study stops at 12 qubits. The device read-outs of Sec. VII are the non-oracle counterpart.

Pre-registration. Every hyperparameter — p , shots, jobs, α , the initial radius, the penalty μ — was fixed on a noise model of `ibm_fez` before any quantum processing time was spent (Appendix A). The depth p was subsequently raised by hand for the larger instances; the consequences are discussed in Sec. VIII.

VII. RESULTS

A. Convergence per job

Figure 3 is the central result, and Table IV quantifies it. The upper row shows the exact noiseless approximation ratio of each arm’s incumbent as a function of the number of jobs spent; the lower row shows what the device reported.

The pattern is consistent across all four instances, and the number of jobs COBYLA needs to catch up grows along the sequence. We are cautious about reading that growth as a size effect: the batch width $M = 4p + 2$ and COBYLA’s initial-model cost $2p + 1$ both grow with p , and p was raised alongside n , so instance size and batch width are confounded here. The claim we make is the weaker and safer one — the ordering holds at every size tested, and at $n = 8$ it survives four independent initializations (Sec. VIIF). After a single job the batched search is already at a noiseless ratio of 0.808, 0.847, 0.888 and 0.915 for

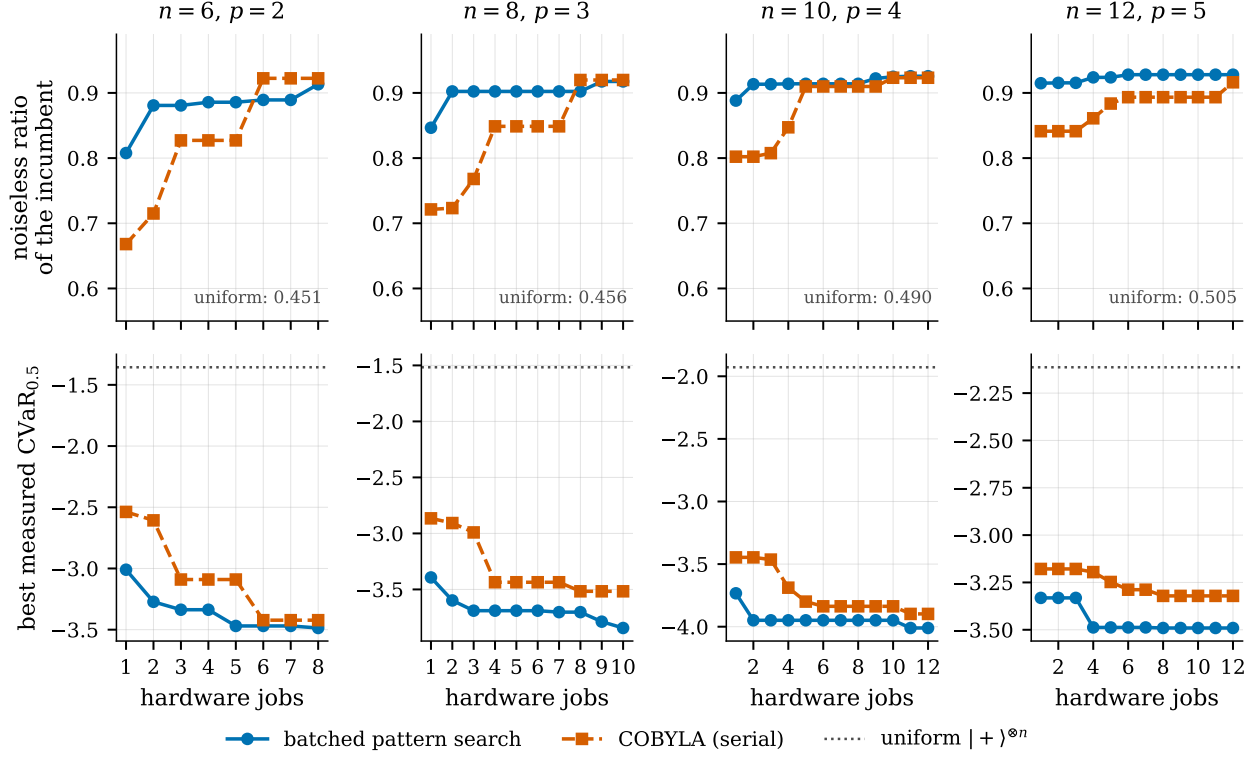


FIG. 3. Optimization on `ibm_fez` under a matched budget of jobs and shots. **Upper:** exact noiseless approximation ratio of the incumbent parameters, recomputed classically after every job — the quality of the point the optimizer has found, independent of how well the device can realize it. **Lower:** best $\text{CVaR}_{0.5}$ actually measured on the device. Both arms receive the same number of jobs and the same shots per job; the batched arm spends a job on $4p + 2$ parameter vectors, the serial arm on one.

$n = 6, 8, 10, 12$ — a quality the serial arm first matches only at job 3, 4, 5 and 12 respectively. Measured by the jobs each arm needs to reach 95% of its own final value, the batched search takes one or two and COBYLA five to eight. At $n = 10$ the serial arm reaches the batched arm’s one-job quality only at job 5, and at $n = 12$ only at the last job of its budget. The parameters each arm actually *returns* for the deep read-out — not the same thing as the best point it ever touched — are worth 0.926 against 0.883 at $n = 10$ and 0.923 against 0.891 at $n = 12$ (Table V).

The mechanism is visible in the lower row of Fig. 3. The serial traces are jagged: COBYLA spends its early jobs building a linear model from single points, and several of those points are worse than where it started. The batched traces are close to monotone,

TABLE IV. Job efficiency on hardware. “COBYLA needs” is the number of jobs after which the serial arm first reaches the quality the batched arm had after its *first* job. “Jobs to 95%” is the number of jobs each arm needs to reach 95% of its own final noiseless ratio.

n	p	jobs/arm	after one BPS job		jobs to 95%		best so far	
			r_{BPS}	COBYLA needs	BPS	COBYLA	r_{BPS}	r_{COB}
6	2	8	0.808	3	2	6	0.913	0.922
8	3	10	0.847	4	2	8	0.918	0.920
10	4	12	0.888	5	1	5	0.926	0.923
12	5	12	0.915	12	1	5	0.928	0.916

TABLE V. Final read-out on `ibm_fez`. ρ_s is the Spearman rank correlation between the measured probability of a feasible portfolio and its exact cost; $\rho_s^{\gamma=0}$ is the same quantity for the noise-floor control.

n	approximation ratio feasible				feasible mass		$\frac{p(\text{opt} \text{feas})}{\text{uniform}}$	rank of opt.	rank corr.	
	BPS	COBYLA	$\gamma=0$	uniform	BPS	uniform			ρ_s	$\rho_s^{\gamma=0}$
6	0.826	0.836	0.446	0.451	0.507	0.312	10.7	1/20	−0.97	+0.07
8	0.772	0.740	0.451	0.456	0.428	0.273	19.2	1/70	−0.88	+0.07
10	0.681	0.662	0.494	0.490	0.321	0.246	16.8	1/252	−0.74	−0.08
12	0.614	0.601	0.506	0.505	0.278	0.226	11.0	2/924	−0.45	−0.00

because each job contains its own incumbent and a stencil around it, and the acceptance test rejects moves that are not larger than shot noise. The batched arm is not finding better optima — at $n = 6$ and 8 the two arms end within 0.01 of each other — it is finding them sooner — one or two jobs against five to eight to reach 95% of its own plateau — and that is precisely the currency that Sec. IV shows to be scarce.

B. What the device delivered

Figure 4 and Table V report the deep read-out at each arm’s best parameters, against the $\gamma = 0$ control and the uniform baseline.

Three observations. First, the device output is well separated from its own noise floor at

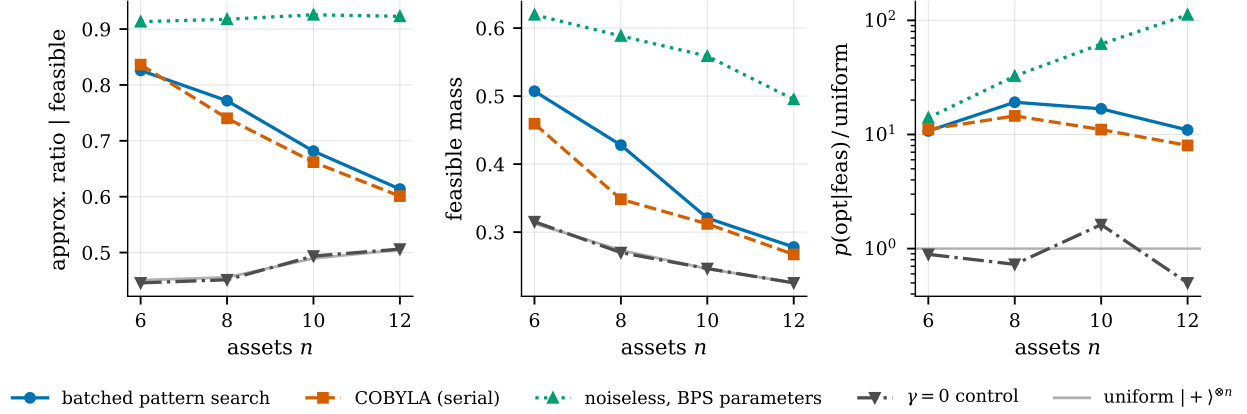


FIG. 4. Final read-out quality against instance size, at 8192 shots. The $\gamma = 0$ control runs a circuit of identical gate count and depth whose ideal output is $|+\rangle^{\otimes n}$; it tracks the uniform baseline at every size, which is what licenses reading the separation between the measured curves and that baseline as signal. The right panel is on a logarithmic scale; the downward arrow on the $\gamma = 0$ control at $n = 12$ marks an upper bound, that circuit having never sampled the optimum in 8192 shots.

every size: the batched arm reaches a conditional approximation ratio of 0.826, 0.772, 0.681 and 0.614 against a $\gamma = 0$ control at 0.446, 0.451, 0.494 and 0.506, the latter tracking the uniform baseline to within 0.005 in every case. Second, the batched arm’s read-out is better than the serial arm’s at $n = 8, 10$ and 12 and marginally worse at $n = 6$. At $n = 10$ and 12 this tracks the noiseless quality of the returned parameters (0.926 against 0.883, 0.923 against 0.891); at $n = 6$ and 8 the two parameter sets are noiselessly equivalent to within 0.01, so the measured difference there reflects device noise rather than a property of the optimizer and should not be read as one — which is what the replicate study of Sec. VII F is for. Third, and most directly relevant to a user, the probability of sampling the exact optimum conditioned on feasibility is enhanced over uniform by factors of 10.7, 19.2, 16.8 and 11.0.

The separation from the baseline narrows as n grows, from 0.38 at $n = 6$ to 0.11 at $n = 12$. Section VII D shows why, and Sec. VIII argues that a substantial part of the narrowing is a consequence of the circuit depth having been increased along with the instance size, not of the instance size alone.

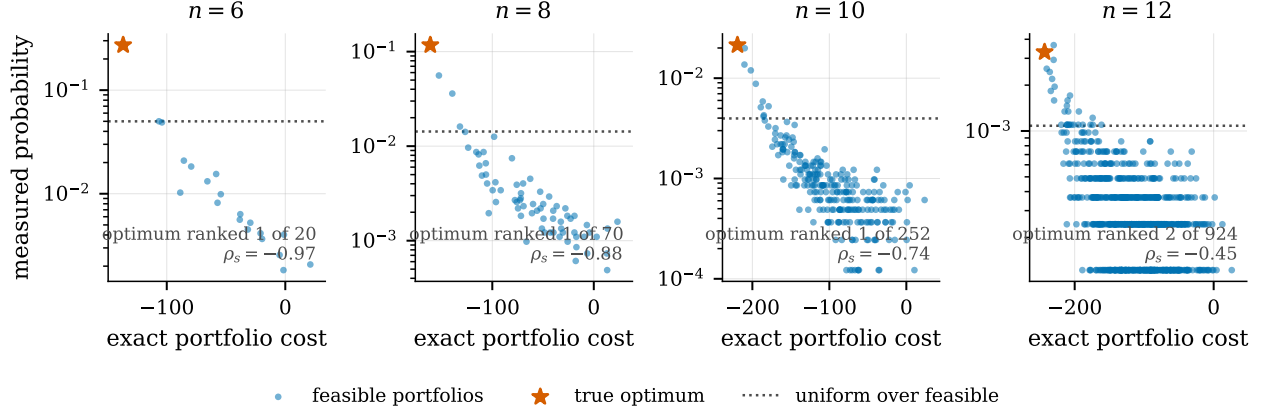


FIG. 5. Measured probability of every feasible portfolio against its exact cost, from the batched arm’s read-out. The star is the true optimum; ρ_s is the Spearman rank correlation. The corresponding correlation for the $\gamma = 0$ control is $\rho_s^{\gamma=0} = +0.07, +0.07, -0.08, -0.00$ for $n = 6, 8, 10, 12$ — consistent with zero.

C. The device ranks portfolios

A user of a portfolio optimizer does not need the exact minimizer; they need a short list. Figure 5 plots, for every feasible portfolio, the probability the device assigned it against its exact cost.

The relationship is monotone and strong: $\rho_s = -0.97, -0.88, -0.74$ and -0.45 . The true optimum is the *most probable* feasible outcome at $n = 6, 8$ and 10 , and the second most probable of 924 candidates at $n = 12$. The same statistic computed on the $\gamma = 0$ control is consistent with zero at every size ($|\rho_s^{\gamma=0}| \leq 0.08$), which is the check that makes the correlation meaningful: a circuit of the same depth on the same qubits, differing only in that its cost-layer angles are zero, carries no information about the cost function whatsoever. Whatever the device is doing at $n = 12$, it is not sorting portfolios by an artefact of the readout chain or the gate set.

D. The device as a depolarizing channel

The measured distributions are well described by a single-parameter model. Writing P_{ideal} for the exact QAOA distribution at the measured parameters, we fit

$$P_{\text{dev}} = \mathcal{R}[(1 - \eta)P_{\text{ideal}} + \eta\mathcal{U}], \quad (9)$$

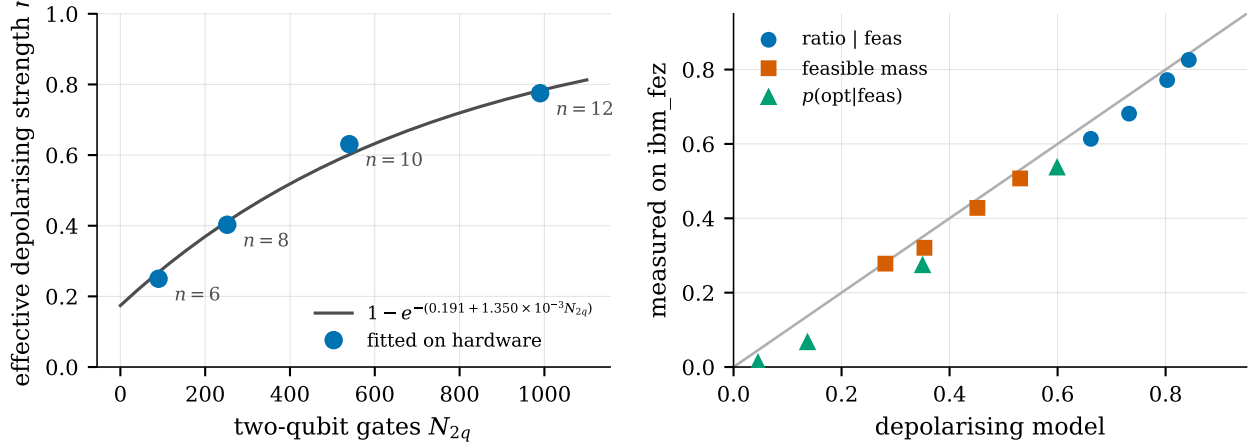


FIG. 6. **Left:** effective depolarizing strength η fitted to the measured mean energy of each instance, against the two-qubit gate count of its circuit; the line is Eq. (10), a two-parameter fit to the four points. **Right:** the model of Eq. (9), with η fitted only to the mean energy, against the measured values of three quantities it was not fitted to.

where $\mathcal{U}$ is uniform and $\mathcal{R}$ applies independent readout bit flips at the measured rate, choosing η so that the model reproduces the measured mean energy — one fitted number per instance. The resulting η are 0.250, 0.403, 0.631 and 0.775 at $N_{2q} = 90, 252, 540$ and 990 two-qubit gates.

These four numbers are described by

$$\eta(N_{2q}) = 1 - \exp[-(0.191 + 1.350 \times 10^{-3} N_{2q})] \quad (10)$$

to within 0.029 (Fig. 6, left). The slope corresponds to an effective error of 1.35×10^{-3} per two-qubit gate, about half the median CZ error of 0.264% reported by the device calibration, which is the expected relationship: a depolarizing channel acting on the *output distribution* is a weaker statement than one acting on the state, and Pauli twirling makes the coherent part of the error stochastic and therefore partially benign. The intercept, 0.191, is the depth-independent contribution — state preparation, single-qubit gates and measurement.

The right panel of Fig. 6 tests the model against quantities it was not fitted to. It reproduces the conditional approximation ratio to within 0.06 and the feasible mass to within 0.04 at every size, and under-predicts the loss in $p(\text{optimum} | \text{feasible})$, which is expected: a global depolarizing channel damages the whole distribution uniformly, whereas correlated and coherent errors damage the sharp peak disproportionately. Equation (10) is

TABLE VI. Depth scan on `ibm.fez`, 8192 shots per point, at classically optimal parameters. First row of each triple: measured approximation ratio. Second, in *italic*: the exact noiseless value at the same parameters. Third: the measured $p(\text{optimum} \mid \text{feasible})$ as a multiple of the uniform value $1/\binom{n}{B}$.

n	$p = 1$	$p = 2$	$p = 3$	$p = 4$	$p = 5$	p^*	p run
6	0.739	0.847	0.845	0.796	0.790	2	2
	<i>0.788</i>	<i>0.930</i>	<i>0.952</i>	<i>0.943</i>	<i>0.963</i>		
	7 $\times$	12 $\times$	12 $\times$	11 $\times$	11 $\times$		
8	0.659	0.781	0.784	0.772	0.747	3	3
	<i>0.692</i>	<i>0.885</i>	<i>0.932</i>	<i>0.959</i>	<i>0.968</i>		
	6 $\times$	19 $\times$	22 $\times$	20 $\times$	19 $\times$		
10	0.619	0.758	0.742	0.713	0.675	2	4
	<i>0.657</i>	<i>0.902</i>	<i>0.933</i>	<i>0.946</i>	<i>0.961</i>		
	8 $\times$	27 $\times$	28 $\times$	25 $\times$	24 $\times$		
12	0.588	0.727	0.710	0.671	0.645	2	5
	<i>0.609</i>	<i>0.882</i>	<i>0.918</i>	<i>0.932</i>	<i>0.952</i>		
	6 $\times$	30 $\times$	30 $\times$	23 $\times$	24 $\times$		

therefore useful as a resource-planning tool — it converts a proposed (n, p) into a predicted output quality before any time is spent — and mildly optimistic as a physical description.

E. Depth is a trade-off, and its optimum is shallow

The circuit depth was increased along with the instance size in the runs above, which leaves the two effects entangled. They can be separated without an optimizer at all. For $n \leq 12$ the noiseless QAOA landscape can be optimized *exactly and classically*, so for each (n, p) we computed the best parameters any optimizer could ever return at that depth, and then simply measured what the device made of them: one job, 8192 shots, 25 jobs in total for 4 sizes $\times$ 5 depths plus a second parameter set at $n = 12$, costing 152 s of metered time. The optimizer is removed from the picture and only the depth–noise trade-off remains.

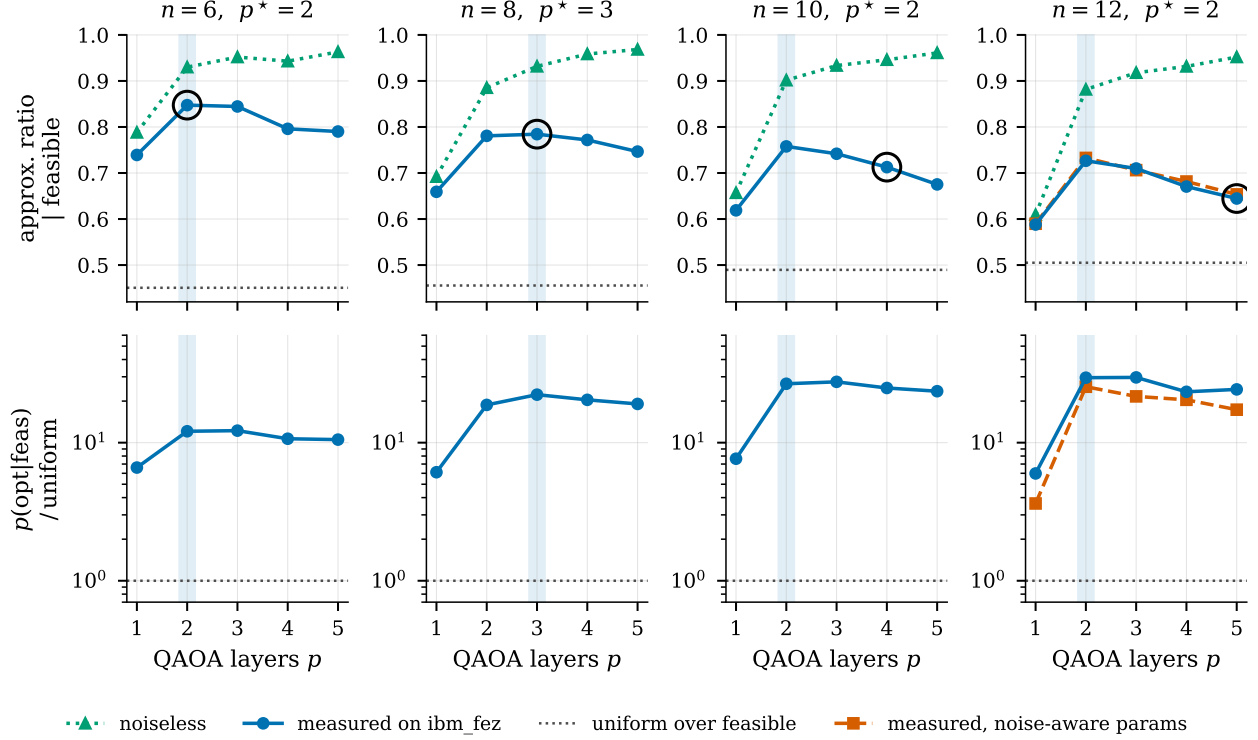


FIG. 7. Approximation ratio against circuit depth at *classically optimal* parameters. The noiseless curve (green) rises with p ; the measured curve (blue) rises, peaks at p^* (shaded), and falls, because $3p\binom{n}{2}$ two-qubit gates eventually cost more fidelity than an extra layer buys. Black rings mark the depth actually used for that size in Sec. VII. At $n = 12$ the orange curve repeats the measurement at parameters optimized against the noise model of Eq. (9) instead of the noiseless landscape. **Lower:** the probability of sampling the exact optimum, conditioned on feasibility, as a multiple of the uniform value $1/\binom{n}{B}$; it peaks at the same depth.

Figure 7 and Table VI show the expected shape, now measured rather than argued. The noiseless ratio rises with p at every size, reaching 0.95–0.97 by $p = 5$; it is monotone at $n = 8, 10$ and 12 , and at $n = 6$ dips by 0.009 between $p = 3$ and $p = 4$, which is the multi-start classical optimizer failing to find the global optimum of a 8-parameter landscape rather than anything physical. The measured ratio does not: it rises steeply from $p = 1$, peaks, and then declines. The peak sits at $p^* = 2, 3, 2, 2$ for $n = 6, 8, 10, 12$, but these are single 8192-shot jobs and at $n = 6$ and $n = 8$ the top two depths differ by only 0.002 and 0.003, which is within shot noise; the statement the data support is that the optimum lies at $p = 2$ or 3 at every size, not which of the two. At $n = 12$ the measured value falls from

0.727 at $p = 2$ to 0.645 at $p = 5$ even though the noiseless value has risen from 0.882 to 0.952 over the same range.

This settles the confound of Sec. VII and is worth stating plainly. The depths run in the main study were $p = 2, 3, 4, 5$; the measured optima are 2, 3, 2, 2. The $n = 6$ and $n = 8$ runs were therefore *at* the device optimum, while the $n = 10$ and $n = 12$ runs were past it, by 0.082 in approximation ratio at $n = 12$. Part of the decline across Fig. 4 is thus a depth artefact rather than a property of the instance — though not all of it: even at each size’s own optimal depth the measured ratio still falls with n , from 0.847 through 0.784 and 0.758 to 0.727.

The second measurement at $n = 12$ answers the obvious follow-up. If the loss comes from running too deep, could an optimizer that *knew* about the noise recover it by choosing different parameters at the same depth? We reoptimized against the model of Eq. (9) at the η predicted by Eq. (10) and measured the result. It changes nothing: the noise-aware parameters differ from the noiseless-optimal ones by at most 0.011 in measured ratio, in either direction, at every depth (orange curve, Fig. 7). The depth is the ceiling, not the parameter choice — consistent with the picture in which the dominant error is a global depolarizing channel, which contracts the whole output distribution toward the uniform one and therefore cannot be steered around by a change of parameters. That noisy devices are limited in exactly this way, and that the limitation tightens with circuit volume, is the subject of a body of rigorous work [42, 43] which our measurement is a small concrete instance of.

The practical rule this yields is concrete: at a given device error rate, choose p by maximizing the product of the noiseless ratio and $1 - \eta(3p\binom{n}{2})$ from Eq. (10), which for `ibm_fez` at these sizes lands on $p = 2$ or 3 and is computable before a single job is submitted.

F. Repeatability across initializations

The comparison of Sec. VII is one trajectory per arm per size. To attach a spread to it we repeated the 8-asset experiment from three further starting points, changing nothing else: same instance, same $p = 3$, same shots, same ten jobs per arm, same 14 336 shots per job for both arms. The starts are the linear ramp at $\Delta t \in \{0.35, 0.50, 0.60, 0.85\}$, with $\Delta t = 0.50$ the run already reported; within a start both arms receive the *same* initial point.

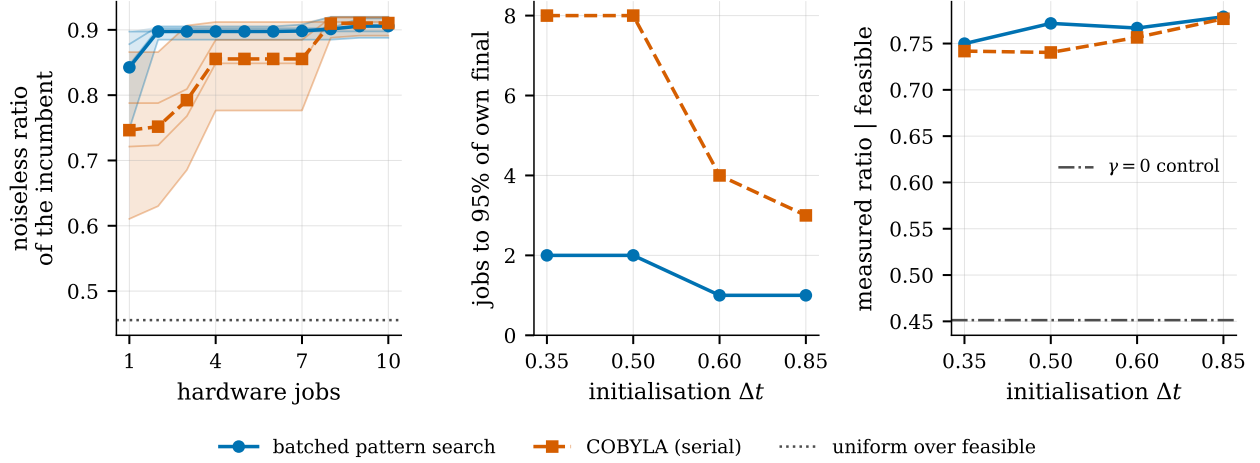


FIG. 8. Four independent initializations at $n = 8$, $p = 3$. **Left:** noiseless approximation ratio of the incumbent against jobs; thin lines are individual starts, markers the mean, bands the full range. **Centre:** jobs each arm needed to reach 95% of its own final value. **Right:** measured approximation ratio of the deep read-out at each arm’s returned parameters, against the $\gamma = 0$ noise floor.

TABLE VII. The 8-asset comparison from four initializations, 66 jobs and 536s of metered time. “COBYLA matches at” is the job at which the serial arm first reaches the quality the batched arm had after its first job; “final, noiseless” is the exact quality of the returned parameters and “read-out” the measured approximation ratio at those parameters. [†] is the run reported in Sec. VII.

	after one job		COBYLA	jobs to 95%		best so far		read-out	
Δt	BPS	COB	matches at	BPS	COB	BPS	COB	BPS	COB
0.35	0.748	0.611	4	2	8	0.888	0.891	0.750	0.742
0.50 [†]	0.847	0.721	4	2	8	0.918	0.920	0.772	0.740
0.60	0.878	0.788	4	1	4	0.920	0.919	0.767	0.757
0.85	0.897	0.866	3	1	3	0.898	0.912	0.779	0.777
mean	0.843	0.746		1.50	5.75	0.906	0.910	0.767	0.754

The ordering holds at every start (Fig. 8, Table VII). The batched search needs 1–2 jobs to reach 95% of its own final value, mean 1.50; COBYLA needs 3–8, mean 5.75. After a single job the batched arm is at 0.843 on average against 0.746 for the serial arm, and COBYLA first matches the batched arm’s one-job quality at job 3, 4, 4 and 8 across the

four starts.

Two further features are worth noting. The *final* parameter values are statistically indistinguishable — 0.906 ± 0.014 for the batched arm against 0.910 ± 0.012 for the serial one — which is the same message as Sec. VII: the two methods find equally good parameters, and differ in how many round trips it takes. The deep read-outs at those parameters (Fig. 8, right) tell the same story with a slight edge to the batched arm: 0.767 ± 0.011 against 0.754 ± 0.015 , with the batched arm ahead at 4 of the four starts, and both far above the $\gamma = 0$ noise floor at 0.451. And the advantage is largest where it matters most, at the poor starting point: at $\Delta t = 0.35$ the batched arm is at 0.748 after one job while the serial arm is at 0.611 and needs eight jobs to converge, whereas at $\Delta t = 0.85$, where the ramp already lands near a good region, both arms converge quickly and the gap narrows to two jobs. A practitioner who cannot pre-tune their initialization — which is the situation whenever the instance is too large to simulate — is exactly the practitioner for whom the batched method buys the most.

VIII. DISCUSSION

A. What the comparison does and does not show

The batched search does not reliably find better optima than COBYLA. Measured by the best point each arm reached, three of the four instances end within 0.01; measured by the parameters each arm actually returned, the two are level at $n = 6$ and 8 and the batched arm is ahead at $n = 10$ and 12. What it does is find them in one or two jobs instead of five to eight, and Sec. IV establishes that jobs are what a cloud user actually spends. The claim is about the shape of the convergence curve near the origin, not about its asymptote, and the practical regime it speaks to is the one where a user has time for a handful of round trips rather than a hundred.

Several limitations bound the strength of the evidence, and we would rather name them than have them found.

Replication. The four main instances were each run once. The replicate study of Sec. VII F attaches a spread to the 8-asset comparison — 1.50 jobs against 5.75 across four initializations, with the ordering holding at every one — but the 10- and 12-asset results are single

trajectories, and the differences in *final* quality there (0.043 and 0.032 in the returned parameters) should be read against the replicate spread of ± 0.01 rather than as precise.

Sequential arms. The two arms ran one after the other rather than interleaved. The drift control of Sec. VI is a single paired repeat per instance, which cannot separate drift from shot noise; it bounds the effect at a few hundredths of a CVaR unit and no more than that. Interleaving would have been cleaner and costs nothing.

Ablations we did not run. Two would sharpen the attribution and each costs one job budget. A *random*-batch arm — M points per job drawn around the same start, greedily tracked — would separate the value of batching per se from the value of the pattern-search stencil. A *serial* pattern search, one point per job with the same acceptance rule, would separate the stencil from the shot-noise tolerance of Algorithm 1, which is a noise-robustness property COBYLA has no analogue of. As designed, this experiment measures the package, not its parts.

A batchable competitor. COBYLA was chosen because it was the strongest *serial* method in the pre-registration study, but as Sec. I notes, SPSA and parameter-shift gradients are themselves batchable. A batched SPSA or a multistart COBYLA packing several concurrent chains into one PUB is the harder target, and we did not run it.

The oracle metric. The per-job curve of Fig. 3 uses the exact noiseless ratio, which no optimizer can compute and which does not exist at a problem size worth solving. It diagnoses the search; it is not a quantity either method returns. The device read-outs of Table V and Table VII are the non-oracle counterpart, and there the two arms are much closer.

Selection bias in the measured traces. The lower row of Fig. 3 plots a running minimum of measured CVaR. The batched arm takes that minimum over M values per job and the serial arm over one, so the batched trace carries a winner’s-curse bias downward. This is one reason the separation is much larger there than in the independently measured deep read-outs, and the read-outs are the number to trust.

B. Depth and instance size were varied together

The circuit depth was increased with the instance size, $p = 2, 3, 4, 5$ for $n = 6, 8, 10, 12$, so the decline in measured quality across Fig. 4 confounds two effects. The depth scan of

Sec. VII E separates them by direct measurement: the device optimum sits at $p^* = 2, 3, 2, 2$, so the $n = 6$ and $n = 8$ runs were at that optimum and the $n = 10$ and $n = 12$ runs were beyond it. Part of the decline is therefore a depth artefact — worth 0.082 in approximation ratio at $n = 12$ — and part is genuine, since the ratio at each size’s own best depth still falls from 0.847 to 0.727 as n grows. Repeating the $n = 12$ optimization at $p = 2$ rather than $p = 5$ is the obvious next experiment; on the evidence of Table VI it would raise the final read-out substantially, and it would not change the job-efficiency comparison, which is a statement about the optimizer at fixed circuit.

C. Consequences beyond QAOA

The cost model of Eq. (8) is a property of the access mode, not of the algorithm. Any variational procedure driven through a queue — a chemistry eigensolver, a quantum kernel evaluation, a circuit-parameter calibration, an error-mitigation sweep — pays the same fixed charge per round trip and can, in principle, fill the same PUB. Three consequences seem general.

First, *benchmark against a job budget*. Reporting convergence against function evaluations makes a serial and a batched optimizer look comparable when their hardware costs differ by an order of magnitude. A plot against submitted jobs is the honest one on cloud hardware, and it is a different plot. The application-oriented benchmarking effort of Ref. [44] already reports execution time including cloud overheads and is the natural home for a job-count axis alongside it.

Second, *prefer optimizers whose next batch is knowable*. This is a design criterion distinct from sample efficiency, and it admits degrees. Pattern search names $4p + 2$ points per round trip; parameter-shift gradients [9, 10] name $4p$; SPSA [8] and its quantum-natural variant [45] name two; batched Bayesian and information-sharing schemes [46] name as many as one chooses; model-based trust-region methods [14] and multistart schemes [15] name a design set. COBYLA, Nelder–Mead and Powell name one. A method that is slightly worse per evaluation but names M points at a time can be an order of magnitude better per job, and the classical direct-search literature worked out how to exploit exactly this parallelism decades ago [13].

Third, *state the cost model with the result*. The community reports shots because shots

were once the binding constraint. On queued hardware in 2026 they are frequently not, and a paper that reports only shots leaves the reader unable to reconstruct what the experiment would cost them.

We note also what the job-budget framing does *not* rescue. Nothing here addresses the fundamental obstacles to variational scaling — barren plateaus [47] and their noise-induced counterpart [43] — and the narrowing separation between the measured curves and the noise floor in Fig. 4 is a reminder that at 12 qubits and 990 two-qubit gates, 78% of the output distribution is already uniform. Job efficiency buys a shorter path to whatever the device can deliver; it does not change what that is.

IX. CONCLUSION

We measured what a job costs on a superconducting quantum processor and found a fixed charge of 3.00 s of metered time — a three-constant law that, once fitted, predicted a further 91 jobs on recalibrated hardware to within 0.0 ms — plus a wall-clock overhead that made 143 minutes of real time buy 34 minutes of quantum processing. Under that cost model we compared a batched pattern search, which spends one job on $4p + 2$ points of the variational landscape, against COBYLA, which spends one job on one point, at matched jobs and matched shots, on four nested cardinality-constrained portfolio problems solved on `ibm_fez` and verified by exhaustive enumeration. The batched search reached in a single job a parameter quality that the serial optimizer required three to twelve jobs to match, with the gap widening as the instance grew. The device returned the exact optimum as its most probable feasible outcome at three of the four sizes and as its second most probable at the fourth, with rank correlations against the exact ordering from $\rho_s = -0.97$ to $\rho_s = -0.45$, while an identically deep $\gamma = 0$ control showed no correlation at all. Two follow-up studies closed the obvious gaps: repeating the 8-asset comparison from four initializations left the ordering intact at every one, and a depth scan at classically optimal parameters located the device’s optimal circuit depth at $p = 2$ – 3 and showed that reoptimizing against the noise model recovers nothing beyond it.

The specific method is simple and, we would argue, is not the point. The point is the unit of account. As long as quantum processors are reached through queues, the number of times an algorithm goes around the loop is the cost that binds, and algorithms should be

designed and reported accordingly.

ACKNOWLEDGMENTS

The author thanks IBM Quantum for access to `ibm_fez`. This work was carried out entirely within the 180 minutes of quantum processing time granted under the IBM Quantum Open Plan promotion of March 2026, and used 22.1 minutes for the four main runs and a further 688 s for the two follow-up studies, 34 minutes in all. The author is a member of the Qiskit Advocate Program. The views expressed are those of the author and do not reflect the official policy or position of IBM or the IBM Quantum team.

DATA AND CODE AVAILABILITY

Every quantum result in this paper is reproduced by six self-contained Jupyter notebooks — one per instance size, plus the replicate and depth-scan studies — released together with the raw data they produced: the complete job log including every IBM job identifier, the measured bitstring counts of every candidate of every job, both optimization traces, the control read-outs and the final metrics. Figures and tables are generated from those files by the included scripts, and every number quoted in the text is emitted by the same scripts rather than transcribed. The archive is available at <https://github.com/<user>/qaoa-job-budget> and archived at Zenodo, DOI <to be assigned>; code is released under the MIT licence and data under CC-BY-4.0.

Appendix A: Selecting the hyperparameters before spending hardware time

Every hyperparameter was fixed on a noise model of `ibm_fez` before any quantum processing time was spent. Full noisy simulation is too slow to sweep — one 12-qubit, $p = 2$ circuit at 2048 shots costs about 80 s with a full Aer noise model — so the sweep used the one-parameter surrogate of Eq. (9), with η fitted per (n, p) against Aer at two parameter points. The surrogate reproduced Aer’s CVaR to about 0.1 in normalized units and its feasible mass to about 0.05.

The sweep covered $p \in \{1, 2, 3\}$, shots per candidate $\in \{512, 1024, 2048\}$, jobs per arm

$\in \{8, 12, 16\}$, $\alpha \in \{0.1, 0.25, 0.5\}$ and initial radius $\in \{0.25, 0.4\}$, filtered to configurations fitting a 300 s budget, at ten random seeds each — 480 configurations in total, and a further confirmation study at forty seeds. It selected $\alpha = 0.5$ at every size, an initial radius of 0.25, and found quality flat in shots between 512 and 2048 and flat in jobs beyond about ten. On the surrogate, COBYLA needed sixteen jobs to match what the batched search had after twelve at $n = 6, 8$ and 10, and did not match it within sixty-four at $n = 12$; the hardware runs of Sec. VII reproduce that ordering, though the surrogate carries neither crosstalk nor drift and its absolute numbers should not be compared with the device’s.

The surrogate selected $p = 2$ at all four sizes, on the grounds that $p = 3$ improved the predicted ratio by at most 0.011 for 50% more two-qubit gates. The depths actually run were $p = 2, 3, 4, 5$, chosen by hand in the belief that a larger instance warrants a deeper circuit. The hardware depth scan of Sec. VIII vindicated the surrogate: the measured optimum is $p = 2$ or 3 at every size. A fake backend carries no crosstalk and no drift, which is precisely why the $\gamma = 0$ and drift controls of Sec. VI are part of every hardware run; but on the one question where its recommendation was overruled, it was right.

Appendix B: Reproducing the experiment

Each notebook is self-contained and runs top to bottom against `ibm_fez` in roughly 100 to 600 s of metered time. A `DRY_RUN` switch replaces the device with the surrogate of Eq. (9) and executes the entire experiment in about a minute at no cost, which is the recommended first step. The ledger of Sec. IV is enforced at run time: before every submission the notebook predicts the metered cost from Eq. (8), switches from the model to the device’s own reported usage after two jobs, and refuses to submit a job that would exceed the configured cap.

Three implementation details caused errors that are silent rather than loud, and are recorded here because they would cost another user a run.

1. `BitArray.get_int_counts()` called with no argument merges the counts of *all* parameter rows of a PUB into one dictionary. A batched run that does this produces plausible output and is entirely meaningless. The row index is mandatory.
2. The parameters of a transpiled circuit are ordered by *name*, not by construction order. The array of parameter values must be assembled by name lookup.

3. An explicit `initial_layout` must be asserted to have survived the pass manager; otherwise the circuit can silently move off the chosen chain.
4. Reading the runtime’s reported usage through `job.metrics()` is version-dependent, and a `try/except` around it will hide a wrong key path behind a plausible fallback. Ours did, on every job of this study, which is why the metered times reported in Sec. IV are modelled rather than measured. If you want the device’s own number, assert that you got one.

The first and the last of these have the same shape: a silent fallback that produces output indistinguishable from success. We would encourage anyone building this kind of ledger to make both loud.

-
- [1] G. S. Ravi, K. N. Smith, P. Gokhale, and F. T. Chong, in *2021 IEEE International Symposium on Workload Characterization (IISWC)* (IEEE, 2021) pp. 39–50, [arXiv:2203.13121](#).
 - [2] G. S. Ravi, K. N. Smith, P. Murali, and F. T. Chong, in *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2021) pp. 301–312, [arXiv:2203.13260](#).
 - [3] W. Wu, Y. Wang, G. Yan, Y. Zhao, B. Zhang, and J. Yan, in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (ACM, 2024) pp. 1–9, [arXiv:2404.07882](#).
 - [4] M. Menickelly, Y. Ha, and M. Otten, *Quantum* **7**, 949 (2023), [arXiv:2201.13438](#).
 - [5] M. J. D. Powell, in *Advances in Optimization and Numerical Analysis*, Mathematics and Its Applications, Vol. 275, edited by S. Gomez and J.-P. Hennart (Springer, Dordrecht, 1994) pp. 51–67.
 - [6] J. A. Nelder and R. Mead, *The Computer Journal* **7**, 308 (1965).
 - [7] M. J. D. Powell, *The Computer Journal* **7**, 155 (1964).
 - [8] J. C. Spall, *IEEE Transactions on Automatic Control* **37**, 332 (1992).
 - [9] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, *Physical Review A* **98**, 032309 (2018), [arXiv:1803.00745](#).
 - [10] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, *Physical Review A* **99**, 032331 (2019), [arXiv:1811.11184](#).

- [11] V. Torczon, [SIAM Journal on Optimization](#) **7**, 1 (1997).
- [12] T. G. Kolda, R. M. Lewis, and V. Torczon, [SIAM Review](#) **45**, 385 (2003).
- [13] P. D. Hough, T. G. Kolda, and V. J. Torczon, [SIAM Journal on Scientific Computing](#) **23**, 134 (2001).
- [14] K. J. Sung, J. Yao, M. P. Harrigan, N. C. Rubin, Z. Jiang, L. Lin, R. Babbush, and J. R. McClean, [Quantum Science and Technology](#) **5**, 044008 (2020), [arXiv:2005.11011](#).
- [15] R. Shaydulin, I. Safro, and J. Larson, in *2019 IEEE High Performance Extreme Computing Conference (HPEC)* (IEEE, 2019) pp. 1–8, [arXiv:1905.08768](#).
- [16] V. Akshay, D. Rabinovich, E. Campos, and J. Biamonte, [Physical Review A](#) **104**, L010401 (2021), [arXiv:2103.11976](#).
- [17] A. Galda, X. Liu, D. Lykov, Y. Alexeev, and I. Safro, in *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2021) pp. 171–180, [arXiv:2106.07531](#).
- [18] S. H. Sureshbabu, D. Herman, R. Shaydulin, J. Basso, S. Chakrabarti, Y. Sun, and M. Pistoia, [Quantum](#) **8**, 1231 (2024), [arXiv:2305.15201](#).
- [19] H. Markowitz, [The Journal of Finance](#) **7**, 77 (1952).
- [20] D. J. Egger, C. Gambella, J. Marecek, S. McFaddin, M. Mevissen, R. Raymond, A. Simonetto, S. Woerner, and E. Yndurain, [IEEE Transactions on Quantum Engineering](#) **1**, 1 (2020), [arXiv:2006.14510](#).
- [21] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, [Nature Reviews Physics](#) **5**, 450 (2023), [arXiv:2307.11230](#).
- [22] M. Hodson, B. Ruck, H. Ong, D. Garvin, and S. Dulman, Portfolio rebalancing experiments using the quantum alternating operator ansatz (2019), [arXiv:1911.05296 \[quant-ph\]](#).
- [23] G. Buonaiuto, F. Gargiulo, G. De Pietro, M. Esposito, and M. Pota, [Scientific Reports](#) **13**, 19434 (2023).
- [24] A. Lucas, [Frontiers in Physics](#) **2**, 5 (2014), [arXiv:1302.5843](#).
- [25] F. Glover, G. Kochenberger, and Y. Du, [4OR](#) **17**, 335 (2019), [arXiv:1811.11538](#).
- [26] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, and S. Woerner, [Quantum](#) **4**, 256 (2020), [arXiv:1907.04769](#).
- [27] D. Amaro, C. Modica, M. Rosenkranz, M. Fiorentini, M. Benedetti, and M. Lubasch, [Quantum Science and Technology](#) **7**, 015021 (2022), [arXiv:2106.10055](#).
- [28] I. D. Kivlichan, J. McClean, N. Wiebe, C. Gidney, A. Aspuru-Guzik, G. K.-L. Chan, and

- R. Babbush, *Physical Review Letters* **120**, 110501 (2018), [arXiv:1711.04789](#).
- [29] B. O’Gorman, W. J. Huggins, E. G. Rieffel, and K. B. Whaley, Generalized swap networks for near-term quantum computing (2019), [arXiv:1905.05118 \[quant-ph\]](#).
- [30] J. Weidenfeller, L. C. Valor, J. Gacon, C. Tornow, L. Bello, S. Woerner, and D. J. Egger, *Quantum* **6**, 870 (2022), [arXiv:2202.03459](#).
- [31] IBM Quantum, Estimate job run time — IBM Quantum Documentation, <https://quantum.cloud.ibm.com/docs/en/guides/estimate-job-run-time> (2026), accessed 20 August 2026.
- [32] IBM Quantum, Plans overview — IBM Quantum Documentation, <https://quantum.cloud.ibm.com/docs/en/guides/plans-overview> (2026), accessed 20 August 2026.
- [33] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, *Physical Review X* **10**, 021067 (2020), [arXiv:1812.01041](#).
- [34] S. H. Sack and M. Serbyn, *Quantum* **5**, 491 (2021), [arXiv:2101.05742](#).
- [35] D. J. Egger, J. Mareček, and S. Woerner, *Quantum* **5**, 479 (2021), [arXiv:2009.10095](#).
- [36] W. Lavrijsen, A. Tudor, J. Müller, C. Iancu, and W. de Jong, in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2020) pp. 267–277, [arXiv:2004.03004](#).
- [37] X. Bonet-Monroig, H. Wang, D. Vermetten, B. Senjean, C. Moussa, T. Bäck, V. Dunjko, and T. E. O’Brien, *Physical Review A* **107**, 032407 (2023), [arXiv:2111.13454](#).
- [38] L. Viola, E. Knill, and S. Lloyd, *Physical Review Letters* **82**, 2417 (1999).
- [39] N. Ezzell, B. Pokharel, L. Tewala, G. Quiroz, and D. A. Lidar, *Physical Review Applied* **20**, 064027 (2023), [arXiv:2207.03670](#).
- [40] J. J. Wallman and J. Emerson, *Physical Review A* **94**, 052325 (2016), [arXiv:1512.01098](#).
- [41] K. Temme, S. Bravyi, and J. M. Gambetta, *Physical Review Letters* **119**, 180509 (2017), [arXiv:1612.02058](#).
- [42] D. Stilck França and R. García-Patrón, *Nature Physics* **17**, 1221 (2021), [arXiv:2009.05532](#).
- [43] S. Wang, E. Fontana, M. Cerezo, K. Sharma, A. Sone, L. Cincio, and P. J. Coles, *Nature Communications* **12**, 6961 (2021), [arXiv:2007.14384](#).
- [44] T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaie, C. H. Baldwin, K. Mayer, and T. Proctor, *IEEE Transactions on Quantum Engineering* **4**, 1 (2023), [arXiv:2110.03137](#).
- [45] J. Gacon, C. Zoufal, G. Carleo, and S. Woerner, *Quantum* **5**, 567 (2021), [arXiv:2103.09232](#).
- [46] C. N. Self, K. E. Khosla, A. W. R. Smith, F. Sauvage, P. D. Haynes, J. Knolle, F. Mintert,

- and M. S. Kim, [npj Quantum Information](#) **7**, 116 (2021), [arXiv:2103.16161](#).
- [47] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, [Nature Communications](#) **9**, 4812 (2018), [arXiv:1803.11173](#).